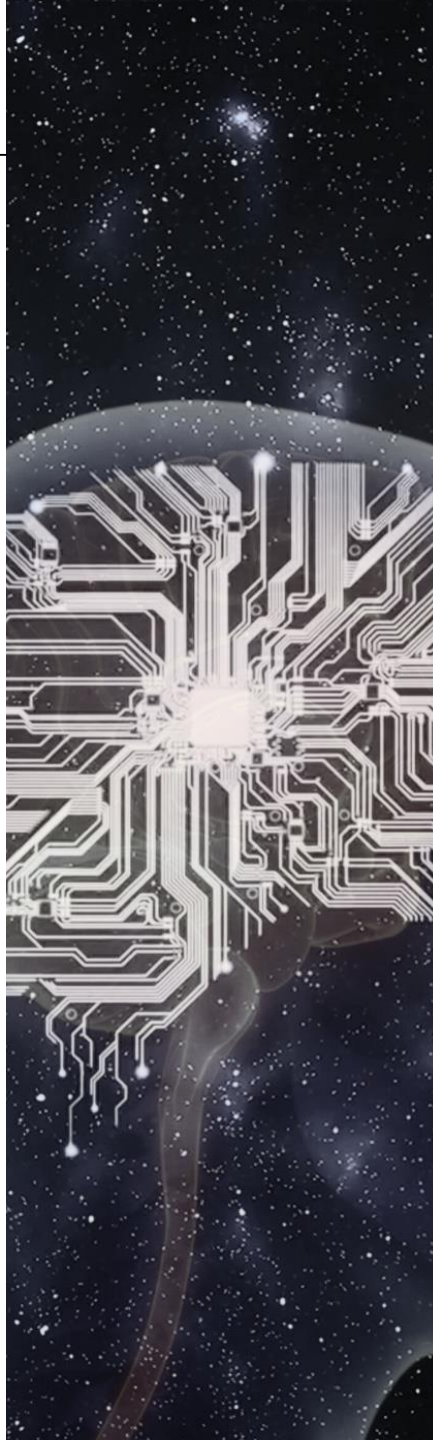
Assembly Programming

9. Strings and Arrays



String Primitive Instructions

In 32-bit mode, each instruction in Table 9-1 implicitly uses ESI, EDI, or both registers to address memory. References to the accumulator imply the use of AL, AX, or EAX, depending on the instruction data size. String primitives execute efficiently because they automatically repeat and increment array indexes.

Table 9-1 String Primitive Instructions.

Instruction	Description
MOVSB, MOVSW, MOVSD	Move string data: Copy data from memory addressed by ESI to memory addressed by EDI.
CMPSB, CMPSW, CMPSD	Compare strings: Compare the contents of two memory locations addressed by ESI and EDI.
SCASB, SCASW, SCASD	Scan string: Compare the accumulator (AL, AX, or EAX) to the contents of memory addressed by EDI.
STOSB, STOSW, STOSD	Store string data: Store the accumulator contents into memory addressed by EDI.
LODSB, LODSW, LODSD	Load accumulator from string: Load memory addressed by ESI into the accumulator.

Using a Repeat Prefix

By itself, a string primitive instruction processes only a single mem ory value or pair of values. If you add a repeat prefix, the instruction repeats, using ECX as a counter. The repeat prefix permits you to process an entire array using a single instruction.

REP	Repeat while ECX > 0
REPZ, REPE	Repeat while the Zero flag is set and ECX > 0
REPNZ, REPNE	Repeat while the Zero flag is clear and ECX > 0

Example: Copy a String

```
cld                             ; clear direction flag
mov   esi,OFFSET string1        ; ESI points to source
mov   edi,OFFSET string2        ; EDI points to target
mov   ecx,10                    ; set counter to 10
rep   movsb                     ; move 10 bytes
```

If ECX > 0, ECX is decremented and the instruction repeats

ESI and EDI are automatically incremented when MOVSB repeats. This behavior is controlled by the CPU's Direction flag.

Direction Flag

String primitive instructions increment or decrement ESI and EDI based on the state of the Direction flag (see Table 9-2). The Direction flag can be explicitly modified using the CLD and STD instructions:

```
CLD         ; clear Direction flag (forward direction)
STD         ; set Direction flag (reverse direction)
```

TABLE 9-2 Direction Flag Usage in String Primitive Instructions.

Value of the Direction Flag	Effect on ESI and EDI	Address Sequence
Clear	Incremented	Low-high
Set	Decremented	High-low

MOVSB, MOVSW, and MOVSD

The MOVSB, MOVSW, and MOVSD instructions copy data from the memory location pointed to by ESI to the memory location pointed to by EDI. The two registers are either incremented or decremented automatically (based on the value of the Direction flag):

MOVSB	Move (copy) bytes
MOVSW	Move (copy) words
MOVSD	Move (copy) doublewords

Instruction	Value Added or Subtracted from ESI and EDI
MOVSB	1
MOVSW	2
MOVSD	4

Example: Copy Doubleword Array

```
.data
source DWORD 20 DUP(0FFFFFFFFh)
target DWORD 20 DUP(?)
.code
cld                                ; direction = forward
mov   ecx,LENGTHOF source          ; set REP counter
mov   esi,OFFSET source            ; ESI points to source
mov   edi,OFFSET target            ; EDI points to target
rep   movsd                        ; copy doublewords
```

CMPSB, CMPSW, and CMPSD

The CMPSB, CMPSW, and CMPSD instructions each compare a memory operand pointed to by ESI to a memory operand pointed to by EDI

CMPSB	Compare bytes
CMPSW	Compare words
CMPSD	Compare doublewords

Example: Comparing Doublewords

Suppose you want to compare a pair of doublewords using CMPSD.

```
.data
source DWORD 1234h
target DWORD 5678h
.code
mov   esi,OFFSET source
mov   edi,OFFSET target
cmpsd                          ; compare doublewords
ja    L1                       ; jump if source > target
```

String Primitive Instructions

To compare multiple doublewords, clear the Direction flag (forward direction), initialize ECX as a counter, and use a repeat prefix with CMPSD

```
mov   esi,OFFSET source
mov   edi,OFFSET target
cld                              ; direction = forward
mov   ecx,LENGTHOF source        ; repetition counter
repe  cmpsd                      ; repeat while equal
```

The REPE prefix repeats the comparison, incrementing ESI and EDI automatically until ECX equals zero or a pair of doublewords is found to be different.

SCASB, SCASW, and SCASD

The SCASB, SCASW, and SCASD instructions compare a value in AL/AX/EAX to a byte, word, or doubleword, respectively, addressed by EDI.

Scan for a Matching Character

In the following example we search the string alpha, looking for the letter F.

```
.data
alpha BYTE "ABCDEFGH",0
.code
mov   edi,OFFSET alpha          ; EDI points to the string
mov   al,'F'                    ; search for the letter F
mov   ecx,LENGTHOF alpha        ; set the search count
cld                             ; direction = forward
repne scasb                     ; repeat while not equal
jnz   quit                      ; quit if letter not found
dec   edi                       ; found: back up EDI
```

STOSB, STOSW, and STOSD

The STOSB, STOSW, and STOSD instructions store the contents of AL/AX/EAX, respectively, in memory at the offset pointed to by EDI.

```
.data
Count = 100
string1 BYTE Count DUP(?)
.code
mov   al,0FFh                    ; value to be stored
mov   edi,OFFSET string1         ; EDI points to target
mov   ecx,Count                  ; character count
cld                              ; direction = forward
rep   stosb                      ; fill with contents of AL
```

initializes each byte in string1 to 0FFh

LODSB, LODSW, and LODSD

The LODSB, LODSW, and LODSD instructions load a byte or word from memory at ESI into AL/AX/EAX, respectively

```
; Multiply an Array                          (Mult.asm)

; This program multiplies each element of an array
; of 32-bit integers by a constant value.

INCLUDE Irvine32.inc
.data
array DWORD 1,2,3,4,5,6,7,8,9,10        ; test data
multiplier DWORD 10                     ; test data

.code
main PROC
     cld                                ; direction = forward
     mov   esi,OFFSET array             ; source index
     mov   edi,esi                      ; destination index
     mov   ecx,LENGTHOF array           ; loop counter
L1:  lodsd                              ; load [ESI] into EAX
     mul   multiplier                   ; multiply by a value
     stosd                              ; store EAX into [EDI]
     loop  L1

     exit
main ENDP
END main
```

multiplies each element of a double word array by a constant value.

In this section, we will demonstrate several procedures from the Irvine32 library that manipulate null-terminated strings.

Str_compare Procedure

```
INVOKE Str_compare, ADDR string1, ADDR string2
```

It compares the strings in forward order, starting at the first byte. The comparison is case sensitive because ASCII codes are different for uppercase and lowercase letters. The procedure does not return a value, but the Carry and Zero flags can be interpreted as shown in Table 9-3, using the string1 and string2 arguments.

Table 9-3 Flags Affected by the Str_compare Procedure.

Relation	Carry Flag	Zero Flag	Branch If True
string1 < string2	1	0	JB
string1 = string2	0	1	JE
string1 > string2	0	0	JA

Selected String Procedures

```
;-----------------------------------------------------
Str_compare PROC USES eax edx esi edi,
     string1:PTR BYTE,
     string2:PTR BYTE
;
; Compares two strings.
; Returns nothing, but the Zero and Carry flags are affected
; exactly as they would be by the CMP instruction.
;-----------------------------------------------------
     mov   esi,string1
     mov   edi,string2
L1:  mov   al,[esi]
     mov   dl,[edi]
     cmp   al,0                ; end of string1?
     jne   L2                  ; no
     cmp   dl,0                ; yes: end of string2?
     jne   L2                  ; no
     jmp   L3                  ; yes, exit with ZF = 1

L2:  inc   esi                 ; point to next
     inc   edi
     cmp   al,dl               ; characters equal?
     je    L1                  ; yes: continue loop
                               ; no: exit with flags set
L3:  ret
Str_compare ENDP
```

CMPSB is most effective when dealing with large strings or arrays of known length.

Str_length Procedure

The Str_length procedure returns the length of a string in the EAX register. When you call it, pass the string's offset.

```
INVOKE Str_length, ADDR myString


Str_length PROC USES edi,
     pString:PTR BYTE           ; pointer to string
     mov edi,pString
     mov eax,0                  ; character count

L1:  cmp BYTE PTR[edi],0        ; end of string?
     je  L2                     ; yes: quit
     inc edi                    ; no: point to next
     inc eax                    ; add 1 to count
     jmp L1
L2:  ret
Str_length ENDP
```

Str_copy Procedure

The Str_copy procedure copies a null-terminated string from a source location to a target location. Before calling this procedure, you must make sure the target operand is large enough to hold the copied string.

```
INVOKE Str_copy, ADDR source, ADDR target
;-----------------------------------------------------
Str_copy PROC USES eax ecx esi edi,
    source:PTR BYTE,            ; source string
    target:PTR BYTE             ; target string
;
; Copies a string from source to target.
; Requires: the target string must contain enough
;           space to hold a copy of the source string.
;-----------------------------------------------------
    INVOKE Str_length,source    ; EAX = length source
    mov   ecx,eax               ; REP count
    inc   ecx                   ; add 1 for null byte
    mov   esi,source
    mov   edi,target
    cld                         ; direction = forward
    rep   movsb                 ; copy the string
    ret
Str_copy ENDP
```

Str_trim Procedure

The Str_trim procedure removes all occurrences of a selected trailing character from a null terminated string.

```
INVOKE Str_trim, ADDR string, char_to_trim
```

1. The string is empty.
2. The string contains other characters followed by one or more trailing characters, as in "Hello##".
3. The string contains only one character, the trailing character, as in "#".
4. The string contains no trailing character, as in "Hello" or "H".
5. The string contains one or more trailing characters followed by one or more nontrailing characters, as in "#H" or "###Hello".

```
.data
string_1 BYTE "Hello##",0
.code
INVOKE Str_trim,ADDR string_1,'#'
INVOKE ShowString,ADDR string_1
```

Selected String Procedures

```
;-----------------------------------------------------
; Str_trim
; Remove all occurrences of a given delimiter
; character from the end of a string.
; Returns: nothing
;-----------------------------------------------------
Str_trim PROC USES eax ecx edi,
     pString:PTR BYTE,           ; points to string
     char: BYTE                  ; character to remove

     mov  edi,pString            ; prepare to call Str_length
     INVOKE Str_length,edi       ; returns the length in EAX
     cmp  eax,0                  ; is the length equal to zero?
     je   L3                     ; yes: exit now
     mov  ecx,eax                ; no: ECX = string length
     dec  eax
     add  edi,eax                ; point to last character

L1:  mov  al,[edi]               ; get a character
     cmp  al,char                ; is it the delimiter?
     jne  L2                     ; no: insert null byte
     dec  edi                    ; yes: keep backing up
     loop L1                     ; until beginning reached

L2:  mov  BYTE PTR [edi+1],0     ; insert a null byte
L3:  ret
Stmr_trim ENDP
```

Table 9-4 Testing the Str_trim Procedure with a # Delimiter Character.

Input String	Expected Modified String
"Hello##"	"Hello"
"#"	"" (empty string)
"Hello"	"Hello"
"H"	"H"
"#H"	"#H"

Detailed Description

The algorithm starts at the end of the string and scans back wards, looking for the first nondelimiter character. When it finds one, a null byte is inserted into the string just after the character position

```
ecx = length(str)
if length(str) > 0 then
    edi = length - 1
    do while ecx > 0
      if str[edi] ≠ delimiter then
          str[edi+1] = null
          break
      else
          edi = edi - 1
      end if
      ecx = ecx - 1
    end do
```

Str_ucase Procedure

The Str_ucase procedure converts a string to all uppercase characters. It returns no value.

```
INVOKE Str_ucase, ADDR myString
```

```
;-----------------------------------------------------
; Str_ucase
; Converts a null-terminated string to uppercase.
; Returns: nothing
;-----------------------------------------------------
Str_ucase PROC USES eax esi,
pString:PTR BYTE

    mov   esi,pString
L1:
    mov   al,[esi]                  ; get char
    cmp   al,0                      ; end of string?
    je    L3                        ; yes: quit
    cmp   al,'a'                    ; below "a"?
    jb    L2
    cmp   al,'z'                    ; above "z"?
    ja    L2
    and   BYTE PTR [esi],11011111b  ; convert the char

L2: inc   esi                       ; next char
    jmp   L1

L3: ret
Str_ucase ENDP
```

(See the *Ucase.asm* program for a demonstration of this procedure.)

String Library Demo Program

```
; String Library Demo (StringDemo.asm)
; This program demonstrates the string-handling procedures in
; the book's link library.
INCLUDE Irvine32.inc

.data
string_1   BYTE "abcde////",0
string_2   BYTE "ABCDE",0
msg0       BYTE "string_1 in upper case: ",0
msg1       BYTE "string_1 and string_2 are equal",0
msg2       BYTE "string_1 is less than string_2",0
msg3       BYTE "string_2 is less than string_1",0
msg4       BYTE "Length of string_2 is ",0
msg5       BYTE "string_1 after trimming: ",0
.code
main PROC
            call trim_string
            call upper_case
            call compare_strings
            call print_length
            exit
main ENDP
```

Selected String Procedures

```
trim_string PROC
; Remove trailing characters from string_1.
        INVOKE Str_trim, ADDR string_1, '/'
        mov edx,OFFSET msg5
        call WriteString
        mov edx,OFFSET string_1
        call WriteString
        call Crlf
        ret
trim_string ENDP

upper_case PROC
; Convert string_1 to upper case.
        mov edx,OFFSET msg0
        call WriteString
        INVOKE Str_ucase, ADDR string_1
        mov edx,OFFSET string_1
        call WriteString
        call Crlf
        ret
upper_case ENDP
```

Selected String Procedures

```
compare_strings PROC
; Compare string_1 to string_2.
        INVOKE Str_compare, ADDR string_1, ADDR string_2
        .IF ZERO?
        mov edx,OFFSET msg1
        .ELSEIF CARRY?
        mov edx,OFFSET msg2                 ; string 1 is less than...
        .ELSE
        mov edx,OFFSET msg3                 ; string 2 is less than...
        .ENDIF
        call WriteString
        call Crlf

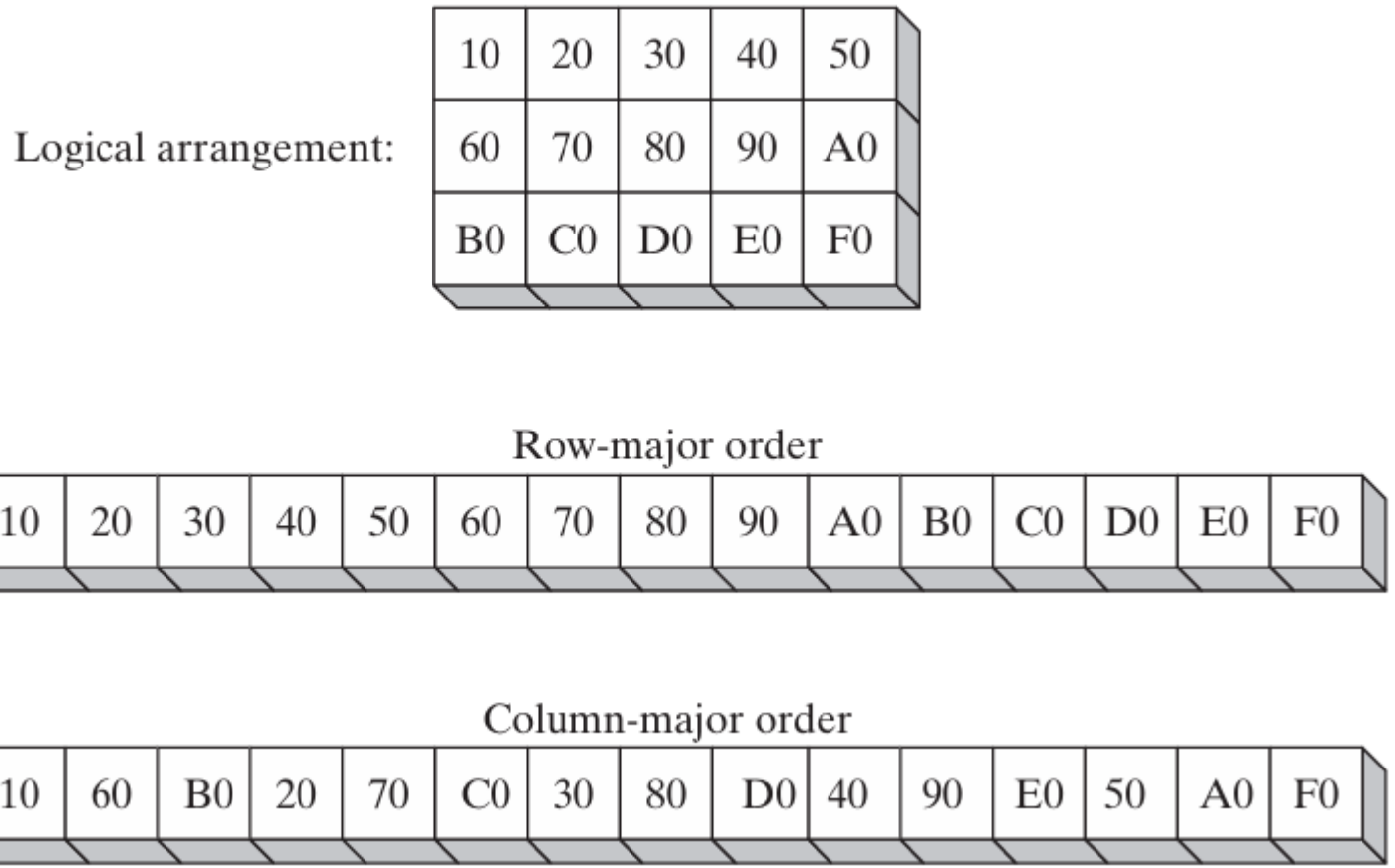
        ret
compare_strings  ENDP
```

```
print_length PROC
; Display the length of string_2.
        mov edx,OFFSET msg4
        call WriteString
        INVOKE Str_length, ADDR string_2
        call WriteDec
        call Crlf
        ret
print_length ENDP
END main
```

Selected String Procedures

Ordering of Rows and Columns

FIGURE 9–1 Row-major and column-major ordering.



Base-Index Operands

A base-index operand adds the values of two registers (called base and index), producing an off set address

```
[base + index]
```

```
.data
array WORD 1000h,2000h,3000h
.code
mov   ebx,OFFSET array
mov   esi,2
mov   ax,[ebx+esi]              ; AX = 2000h

mov   edi,OFFSET array
mov   ecx,4
mov   ax,[edi+ecx]              ; AX = 3000h

mov   ebp,OFFSET array
mov   esi,0
mov   ax,[ebp+esi]              ; AX = 1000h
```

Two-Dimensional Array

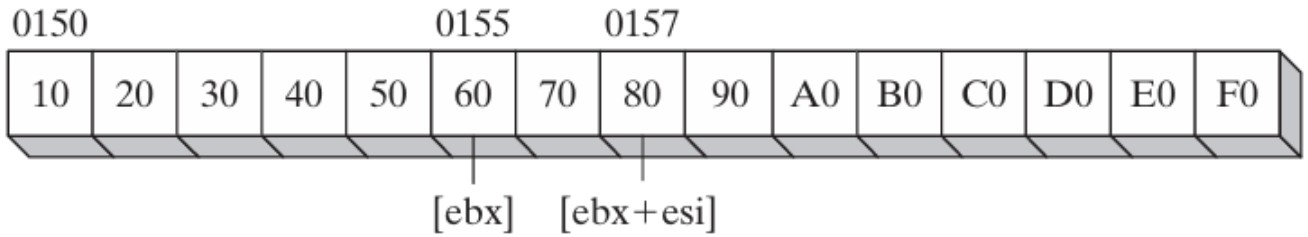
When accessing a two-dimensional array in row-major order, the row offset is held in the base register and the column offset is in the index register.

```
tableB  BYTE   10h,  20h,  30h,  40h,  50h
Rowsize = ($ - tableB)
        BYTE   60h,  70h,  80h,  90h, 0A0h
        BYTE  0B0h, 0C0h, 0D0h, 0E0h, 0F0h

row_index = 1
column_index = 2

mov  ebx,OFFSET tableB          ; table offset
add  ebx,RowSize * row_index    ; row offset
mov  esi,column_index
mov  al,[ebx + esi]             ; AL = 80h
```

FIGURE 9–2 Addressing an array with a base-index operand.



Calculating a Row Sum

```
;-----------------------------------------------------
; calc_row_sum
; Calculates the sum of a row in a byte matrix.
; Receives: EBX = table offset, EAX = row index,
; ECX = row size, in bytes.
; Returns: EAX holds the sum.
;-----------------------------------------------------
calc_row_sum PROC USES ebx ecx edx esi
        mul ecx                         ; row index * row size
        add ebx,eax                     ; row offset
        mov eax,0                       ; accumulator
        mov esi,0                       ; column index
L1:     movzx edx,BYTE PTR[ebx + esi]   ; get a byte
        add eax,edx                     ; add to accumulator
        inc esi                         ; next byte in row
        loop L1
        ret
calc_row_sum ENDP
```

calculates the sum of a selected row in a matrix of 8-bit integers:

Scale Factors

If you're writing code for an array of WORD, multiply the index operand by a scale factor of 2.

```
tableW  WORD   10h,  20h,  30h,  40h,  50h
RowsizeW = ($ - tableW)
        WORD   60h,  70h,  80h,  90h, 0A0h
        WORD  0B0h, 0C0h, 0D0h, 0E0h, 0F0h
.code
row_index = 1
column_index = 2
mov   ebx,OFFSET tableW                ; table offset
add   ebx,RowSizeW * row_index         ; row offset
mov   esi,column_index
mov   ax,[ebx + esi*TYPE tableW]       ; AX = 0080h


tableD DWORD 10h, 20h, ...etc.
.code
mov   eax,[ebx + esi*TYPE tableD]
```

Base-Index-Displacement Operands

A base-index-displacement operand combines a displacement, a base register, an index register, and an optional scale factor to produce an effective address. Displacement can be the name of a variable or a constant expression.

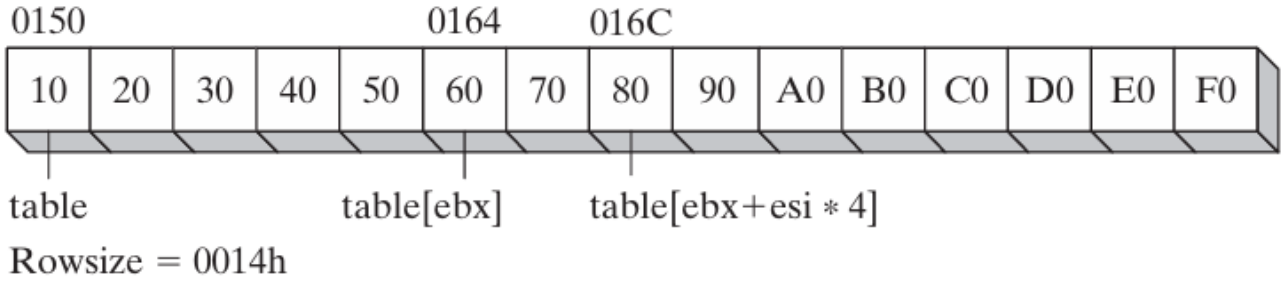
```
[base + index + displacement]
displacement[base + index]
```

Doubleword Array Example

```
tableD DWORD   10h,  20h,  30h,  40h,  50h
Rowsize = ($ - tableD)
       DWORD   60h,  70h,  80h,  90h,  0A0h
       DWORD  0B0h, 0C0h, 0D0h, 0E0h,  0F0h

mov   ebx,Rowsize                ; row index
mov   esi,2                      ; column index
mov   eax,tableD[ebx + esi*TYPE tableD]
```

FIGURE 9–3 Base-index-displacement example.

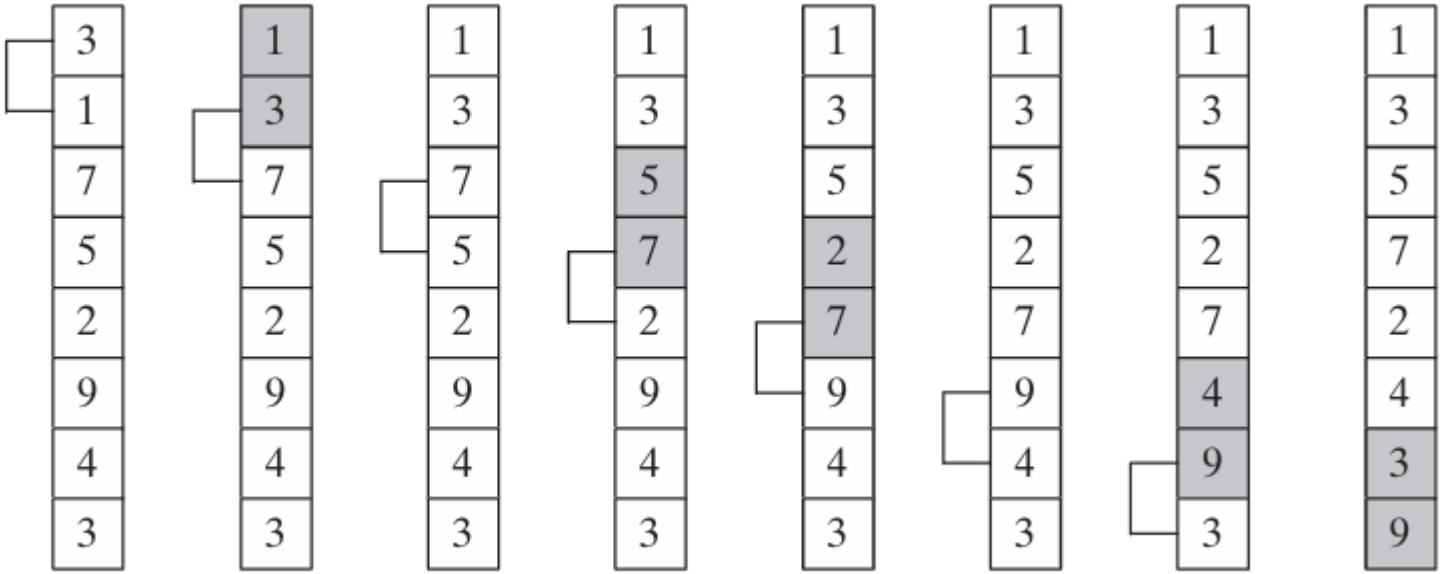


Two-Dimensional Arrays

Bubble Sort

A bubble sort compares pairs of array values, beginning in positions 0 and 1. If the compared values are in reverse order, they are exchanged.
After one pass, the array is still not sorted, but the largest value is now in the highest index position. The outer loop starts another pass through the array. After n-1 passes, the array is guaranteed to be sorted.

FIGURE 9–4 First pass through an array (bubble sort).



(Shaded values have been exchanged)

Pseudocode

```
cx1 = N - 1
while( cx1 > 0 )
{
  esi = addr(array)
  cx2 = cx1
  while( cx2 > 0 )
  {
    if( array[esi] > array[esi+4] )
      exchange( array[esi], array[esi+4] )
    add esi,4
    dec cx2
  }
  dec cx1
}
```

Searching and Sorting Integers Arrays

Assembly Language

```
;-----------------------------------------------------
; BubbleSort
; Sort an array of 32-bit signed integers in ascending
; order, using the bubble sort algorithm.
; Receives: pointer to array, array size
; Returns: nothing
;-----------------------------------------------------
BubbleSort PROC USES eax ecx esi,
        pArray:PTR DWORD,   ; pointer to array
        Count:DWORD         ; array size
        mov ecx,Count
        dec ecx             ; decrement count by 1
L1:     push ecx            ; save outer loop count
        mov esi,pArray      ; point to first value
L2:     mov eax,[esi]       ; get array value
        cmp [esi+4],eax     ; compare a pair of values
        jg  L3              ; if [ESI] <= [ESI+4], no exchange
        xchg eax,[esi+4]    ; exchange the pair
        mov [esi],eax
L3:     add esi,4           ; move both pointers forward
        loop L2             ; inner loop
        pop ecx             ; retrieve outer loop count
        loop L1             ; else repeat outer loop
L4:     ret
BubbleSort ENDP
```

Binary Search

Before beginning the search, ask the user to enter an integer, which we will call searchVal.

1. The range of the array to be searched is indicated by the subscripts named first and last. If f irst last, exit the search, indicating failure to find a match.
2. Calculate the midpoint of the array between array subscripts first and last.
3. Compare searchVal to the integer at the midpoint of the array:
   - If the values are equal, return from the procedure with the midpoint in EAX. This return value indicates that a match has been found in the array.
   - On the other hand, if searchVal is larger than the number at the midpoint, reset first to one position higher than the midpoint.
   - Or, if searchVal is smaller than the number at the midpoint, reset last to one position below the midpoint.
4. Return to Step 1.

Following is a C++ implementation of a binary search function designed to work with an array of signed integers:

```
int BinSearch( int values[], const int searchVal, int count )
{
  int first = 0;
  int last = count - 1;

  while( first <= last )
  {
    int mid = (last + first) / 2;
    if( values[mid] < searchVal )
      first = mid + 1;
    else if( values[mid] > searchVal )
      last = mid - 1;
    else
      return mid;          // success
  }
  return -1;               // not found
}
```

The following listing shows an assembly language implementation of the sample C++ code:

```
;-----------------------------------------------------
; BinarySearch
; Searches an array of signed integers for a single value.
; Receives: Pointer to array, array size, search value.
; Returns: If a match is found, EAX = the array position of the
; matching element; otherwise, EAX = -1.
;-----------------------------------------------------
BinarySearch PROC USES ebx edx esi edi,
        pArray:PTR DWORD,           ; pointer to array
        Count:DWORD,                ; array size
        searchVal:DWORD             ; search value
        LOCAL first:DWORD,          ; first position
        last:DWORD,                 ; last position
        mid:DWORD                   ; midpoint
        mov first,0                 ; first = 0
        mov eax,Count               ; last = (count - 1)
        dec eax
        mov last,eax
        mov edi,searchVal           ; EDI = searchVal
        mov ebx,pArray              ; EBX points to the array
```

Searching and Sorting Integers Arrays

```
L1:                                     ; while first <= last
            mov eax,first
            cmp eax,last
            jg  L5                      ; exit search
; mid = (last + first) / 2
            mov eax,last
            add eax,first
            shr eax,1
            mov mid,eax
; EDX = values[mid]
            mov esi,mid
            shl esi,2                   ; scale mid value by 4
            mov edx,[ebx+esi]           ; EDX = values[mid]
; if ( EDX < searchval(EDI) )
            cmp edx,edi
            jge L2
; first = mid + 1
            mov eax,mid
            inc eax
            mov first,eax
            jmp L4
; else if( EDX > searchVal(EDI) )
```

```
L2:         cmp edx,edi             ; optional

            jle L3
;last = mid - 1
            mov eax,mid
            dec eax
            mov last,eax
            jmp L4
; else return mid
L3:         mov eax,mid             ; value found
            jmp L9                  ; return (mid)
L4:         jmp L1                  ; continue the loop
L5:         mov eax,-1              ; search failed
L9:         ret
BinarySearch ENDP
```

Test Program

- Fills an array with random integers
- Displays the array
- Sorts the array using a bubble sort
- Redisplays the array
- Asks the user to enter an integer
- Performs a binary search for the user's integer (in the array)
- Displays the results of the binary search

Searching and Sorting Integers Arrays

Strings and Arrays

9.1 Introduction

9.2 String Primitive Instructions
- 9.2.1 MOVSB, MOVSW, and MOVSD
- 9.2.2 CMPSB, CMPSW, and CMPSD
- 9.2.3 SCASB, SCASW, and SCASD
- 9.2.4 STOSB, STOSW, and STOSD
- 9.2.5 LODSB, LODSW, and LODSD
- 9.2.6 Section Review

9.3 Selected String Procedures
- 9.3.1 Str_compare Procedure
- 9.3.2 Str_length Procedure
- 9.3.3 Str_copy Procedure
- 9.3.4 Str_trim Procedure
- 9.3.5 Str_ucase Procedure
- 9.3.6 *String Library Demo* Program
- 9.3.7 String Procedures in the Irvine64 Library
- 9.3.8 Section Review

9.4 Two-Dimensional Arrays
- 9.4.1 Ordering of Rows and Columns
- 9.4.2 Base-Index Operands
- 9.4.3 Base-Index-Displacement Operands
- 9.4.4 Base-Index Operands in 64-Bit Mode
- 9.4.5 Section Review

9.5 Searching and Sorting Integer Arrays
- 9.5.1 Bubble Sort
- 9.5.2 Binary Search
- 9.5.3 Section Review

9.6 Java Bytecodes: String Processing (Optional Topic)

9.7 Chapter Summary

9.8 Key Terms and Instructions

9.9 Review Questions and Exercises
- 9.9.1 Short Answer
- 9.9.2 Algorithm Workbench

9.10 Programming Exercises

https://www.asmirvine.com/

Review questions and Exercises

&

Programming exercises

Github upload